

Name - B. Vishnu

Course Code - CSA 1712

Reg. No. 192512410

Course Name - Artificial Intelligence

Assessment - CO1 AT3

1) To effectively manage lighting, temperature, and security in a smart home, the system must process real-time environmental inputs and learn user preferences over time.

- Simple Reflex Agent: This agent operates purely on condition-action rules based on the current percept, ignoring any historical data.

* Function in the system: If the motion sensor detects movement, it instantly turns on the lights. If the temperature hits 25°C , it turns on the AC.

* Limitation: It has no memory. It might turn on the bright lights in the middle of the night just because it detected motion, ignoring the user's sleep cycle.

- Model-Based Reflex Agent: This agent maintains an internal state that keeps track of the world's history and how it evolves over time.

*Function in the system: It remembers context. If motion is detected, it checks its internal state for the "time of day". If it is 2.00 A.M. it triggers a dim nightlight instead of the main lights. If a window is open [tracked via sensors], it will avoid turning on the AC.

- Goal-Based Agent: This agent acts to achieve specific, programmed goals (eg. "maintain house temperature at 22°C ." or "keep the house secure").

*Function in the system: It plans its actions. If the goal is a secure house and the family leaves, it will automatically lock doors, arm the alarm, and turn off unnecessary lights.

*Limitations: It only cares about reaching the goal, not the cost. It might run the AC at maximum power all day to keep the temperature exactly at 22°C , wasting energy.

- Utility-Based Agent: This agent evaluates how "happy" or efficient a state is, using a utility function to balance conflicting goals.

Justification for the Most Effective Approach:

The most effective solution would be a Hybrid Utility-Based Learning Agent.

- Why? → A pure goal based agent can't handle the trade-offs between energy efficiency and comfort.
- Furthermore, because the system must "learn user preferences over time", integrating a Learning Agent architecture allows the system to adjust its utility function. If the user consistently overrides the thermostat to 20°C in the evenings, the agent learns this new preference, updating its decision-making model to maximize user satisfaction, energy efficiency, and security simultaneously.

2) When a robot is placed in an unknown maze, it must navigate without prior knowledge of the layout to find an exit. Uniformed Search strategies like Uniform Cost Search [UCS] and Iterative Deepening Search [IDS] are critical for such pathfinding problems.

Explanation of Search Strategies:

- Uniform Cost Search [UCS]: UCS expands nodes based on the lowest path cost $g(n)$ from the start node rather than depth. In a maze where moving through different terrains [eg. mud vs smooth floor] might consume differently amounts of battery, UCS ensures the robot finds the absolute cheapest path to the exit.
- Iterative Deepening Search [IDS]: IDS repeatedly applies Depth-first Search [DFS] with an increasing depth limit until the goal is found.

Time and Space Complexity:

* Uniform Cost Search [UCS]:

- Time Complexity: $O(b' + [c^* / \epsilon])$, where c^* is the cost of the optimal solution and ϵ is the minimum action cost.
- Space Complexity: $O(b' + [c^* / \epsilon])$
- Advantage: It is optimal and complete
- Disadvantage: It requires massive amounts of memory (space) to store the frontier, frontier, which could overwhelm a small robot's hardware.

* Iterative Deepening Search [IDS]:

- Time Complexity: $O(b^d)$, where b is the branching factor and d is the depth of the shallowest goal.
- Space Complexity: $O(bd)$
- Advantage: Extremely space-efficient [linear memory requirement] while guaranteeing the shortest path is found.
- Disadvantage: Recomputes upper-level nodes multiple times [though asymptotically, the time overhead is minimal].

Relation to Intelligent Agents and Decision Making:

Agent design severely limits or enhances decision-making capabilities in a maze.

- A Simple Reflex Agent would fail because it makes decisions based only on the current percept, inevitably getting trapped.
- A Model-Based Agent could map out dead ends (internal state), avoiding loops, but can't actively plan a route.

∴ The robot must be a Goal-Based Agent.

Justification for Best Combination:

The most effective combination is a Goal Based Agent paired with Iterative Deepening Search (IDS).

⇒ In an unknown maze, the robot does not know how far the exit is or the total state space size.

⇒ IDS allows the goal-based agent to guarantee finding the shortest path to exit without crashing the robot's onboard computer.